

Iqra Digital Library: the v3 rebuild

Retiring Gradio, exposing the RAG engine as a typed API, and putting a TypeScript product interface in front of it — on the DiploTech palette, deployed publicly, at zero monthly cost.

Repo Suleman96/Digitalized-Library-Management-System **Branch** feature/v3-typescript-frontend
From v2 (Gradio 4.x) **Status** P0 + P1 shipped · 110 tests green

PROGRESS — P0 AND P1 ARE DONE

The data layer is extracted and the FastAPI service is live. Verified against the running API on the real 6,810-book catalogue:

TESTS

110

passing, up from 81. Includes 22 new API contract tests and 6 covering the pipeline trace.

HYBRID QUERY

71 ms

BM25 + FAISS + knowledge graph, measured end-to-end through the live endpoint.

KNOWLEDGE GRAPH

69,116

edges across 6,810 nodes, loaded from cache at boot.

Remaining: P2–P6 (frontend, streaming UI, deployment). See the phase list below — completed phases are marked.

THE CALL

The problem with the current build is not that it is written in Python — it is that **the Python is writing HTML**. `recommender.py` returns rendered book cards as strings. `reading_list.py` returns a rendered table. `app.py` holds 1,260 lines of CSS inside a Python string literal. No frontend of any language can be built on that, because there is no data layer to build on.

So the migration is a **split, not a rewrite**. The retrieval engine you built — hybrid BM25 + FAISS over a NetworkX knowledge graph, HyDE, multi-query RRF, cross-encoder re-ranking, a LangGraph ReAct agent across five LLM providers — stays in Python, because FAISS, `sentence-transformers`, and LangGraph have no honest JavaScript equivalent. It gets refactored to return JSON and served by FastAPI. A Next.js 15 + TypeScript app consumes that API and becomes the thing people actually see.

Net result for a reviewer landing on your GitHub: a typed monorepo, an OpenAPI contract, a live URL, and an ML backend that is now legible instead of buried in a 2,724-line UI file.

START HERE — WHAT IS BEING REBUILT, AND WHY

The original app worked, but it was built with a tool called Gradio — designed for researchers to demo an idea quickly, not for building a product people use. Think of it as a very good prototyping kit: fast to get something on screen, but you inherit its look, its layout, and its limits.

The deeper problem was structural. In the old version, the *engine* (the part that finds books) and the *interface* (the part you look at) were fused together. The engine was handing back finished web-page fragments instead of plain information. That is like a chef who refuses to give you ingredients and will only hand over a plated dish — fine, until you want to serve it differently.

So this rebuild separates the kitchen from the dining room. The engine now returns raw information; a new, purpose-built interface decides how to present it.

Nothing clever is being thrown away. The search intelligence — the genuinely hard part, and the part worth showing an employer — is kept exactly as it is. What is being replaced is only the packaging.

01 — What you have today

An audit of the current tree, and the disposition of each piece in v3.

READING THE AUDIT TABLE

Three outcomes for each file. **Keep** means untouched — it already does its job properly. **Split** means the file does two jobs and only one belongs there. **Delete** means it existed only to serve the old interface.

The two numbers underneath tell the whole story: roughly 1,700 lines of genuinely valuable engine survive, and roughly 3,200 lines of packaging go. *Most of what is being deleted is a stylesheet written inside a Python file* — which is as awkward as it sounds.

FILE	LINES	WHAT IT DOES	DISPOSITION
app.py	2,724	Gradio Blocks UI, 6 tabs, ~1,260 lines of inline CSS, 30 event handlers	DELETE Handlers become API routes
recommender.py	525	Book normalisation, Google Books + OpenLibrary clients, <i>and HTML card rendering</i>	SPLIT Keep search, cut <code>_render_card</code>
hybrid_retriever.py	434	BM25 (40%) + FAISS (60%) min-max fusion, NetworkX knowledge graph over authors/categories	KEEP AS-IS
llm_provider.py	406	Claude / OpenAI / Gemini / Groq / Ollama abstraction, HyDE + query-variant generation	KEEP AS-IS
agent.py	194	LangGraph ReAct agent, 5 tools, <code>MemorySaver</code> threads	EXTEND Add token streaming
manager.py	182	CSV ↔ FAISS lifecycle, atomic add/remove	KEEP AS-IS
config.py	165	Frozen <code>Settings</code> dataclass, key resolution	KEEP AS-IS
rag_pipeline.py	152	HyDE, multi-query + RRF, cross-encoder re-rank, graceful degradation	KEEP AS-IS
reading_list.py	141	JSON-backed bookmarks, <i>and</i> <code>to_html()</code>	SPLIT Cut <code>to_html</code>
exporter.py	125	fpdf2 PDF export	KEEP Serve as a download route
tests/ ×5	—	81 passing tests over config, manager, recommender, retriever, RAG	KEEP Add API contract tests

ENGINE, WORTH KEEPING

1,673

lines of retrieval, RAG, and agent code that survive the migration untouched or nearly so.

PRESENTATION, BEING REPLACED

3,249

lines of Gradio UI, inline CSS, and Python-side HTML rendering that v3 deletes.

CURRENT TESTS

81

passing. None of them cover the UI layer, which is why deleting it is low-risk.

That ratio is the whole argument. You are not throwing away work — you are throwing away the *packaging*, which is the part that was never the interesting bit anyway.

02 — The blocker, precisely

Here is the pattern that has to change, taken from the current code:

```
# recommender.py - the backend is a template engine
def _render_card(self, book: dict) -> str:
    return f'<div class="book-card">...'

# reading_list.py
def to_html(self) -> str: ...

# app.py - handlers return HTML strings straight into gr.HTML()
def _do_recommend(...) -> tuple[str, gr.update, list]: ...
def _load_reading_list() -> str: ...
def _concierge_ai_status() -> str: ...
```

Every one of these must return a Python object that serialises to JSON. Until that is true, the frontend language is irrelevant — React would just be receiving pre-rendered markup and dumping it through `dangerouslySetInnerHTML`, which is both an XSS surface and a pointless architecture.

The good news: the canonical book schema already exists.

`BookRecommender._normalise()` at [recommender.py:139](#) already produces exactly the flat dict a TypeScript type wants — `title`, `authors`, `subtitle`, `description`, `thumbnail`, `average_rating`, `ratings_count`, `info_link`, `language`, `published_year`, `num_pages`, `source`, `similarity`. The data model is sound. It is only the last mile that got templated.

The one schema change worth making

Books currently have no stable identifier — the UI selects them by `title` string, which breaks on duplicates and on any title containing punctuation. v3 adds a deterministic `id` (a hash of title + first author) so the frontend can key lists and route to `/book/[id]` properly.

Analytics needs the same treatment

`_build_analytics()` currently renders three Matplotlib PNGs server-side. In v3 it returns plain series data and the frontend draws them — which is faster, interactive, theme-aware, and removes Matplotlib from the server image entirely.

03 — Target architecture

THE NEW SHAPE, IN ONE PARAGRAPH

Three layers, each with one job. **apps/web** is everything you see and click — written in TypeScript, the language of modern web interfaces. **apps/api** is the switchboard: it receives requests from the interface and passes them to the engine. **apps/api/core** is the engine itself, which knows how to find books and nothing at all about websites.

The value of the separation is that each layer can be changed, tested, or replaced without disturbing the others. You could put a mobile app in front of the same engine tomorrow and change nothing underneath.

```
iqra-digital-library/
├── apps/web/                                     ← NEW · Next.js 15 · TypeScript
│   ├── app/
│   │   ├── layout.tsx                          root shell, theme provider, fonts
│   │   ├── page.tsx                            Discovery – the landing surface
│   │   ├── concierge/page.tsx                  streaming agent chat
│   │   ├── catalogue/page.tsx                 paginated browse + filters
│   │   ├── reading-list/page.tsx              saved books
│   │   ├── analytics/page.tsx                 charts
│   │   ├── book/[id]/page.tsx                 detail + similar books
│   │   └── how-it-works/page.tsx              the RAG explainer – see §09
│   ├── components/
│   │   ├── ui/                                shadcn/ui primitives
│   │   ├── book-card.tsx                      replaces _render_card
│   │   ├── search-panel.tsx                   replaces the Gradio sidebar
│   │   ├── pipeline-trace.tsx                 NEW – visualises the RAG stages
│   │   └── chat-stream.tsx                    SSE consumer
│   ├── lib/api.ts                             typed fetch client
│   └── lib/types.ts                           generated from OpenAPI
├── apps/api/                                    ← NEW shell, EXISTING brain
│   ├── main.py                                FastAPI app + CORS
│   ├── routers/
│   │   ├── search.py                          agent.py
│   │   ├── books.py                          reading_list.py
│   │   └── analytics.py                       llm.py
│   ├── schemas.py                             Pydantic models = source of truth
│   ├── deps.py                                singleton lifespan wiring
│   └── core/                                  your existing modules, moved
│       ├── hybrid_retriever.py                unchanged
│       ├── rag_pipeline.py                    unchanged
│       ├── llm_provider.py                    unchanged
│       ├── agent.py                           + streaming
│       ├── manager.py                         unchanged
│       ├── recommender.py                     HTML rendering removed
│       ├── reading_list.py                    to_html removed
│       └── config.py                          unchanged
├── data/books.csv                             ~6,800 books, unchanged
├── docs/                                       architecture + this dossier
└── .github/workflows/                         pytest + tsc + eslint + build
```

Startup wiring moves out of module scope and into a FastAPI `lifespan` handler — the current app builds `HybridRetriever`, loads `all-MiniLM-L6-v2`, and constructs the agent at import time, which makes the module impossible to import in a test without paying a ~15-second model load.

CHOICE	INSTEAD OF	REASON
Next.js 15, App Router	Vite SPA	Server components render book grids without shipping the data twice; file routing gives shareable URLs per book, which the Gradio version cannot do at all
TypeScript, strict: true	JavaScript	The point of the exercise. Types generated from the API's OpenAPI schema, so drift is a build error
Tailwind v4 + shadcn/ui	Hand-written CSS	You already hand-wrote 1,260 lines of CSS once. Tokens in @theme, components you own in-repo, no runtime dependency
TanStack Query	Redux / Zustand	Nearly all state here is server state — caching, dedupe, and background refetch are the actual requirements
Recharts	Matplotlib PNGs	Interactive, theme-aware, and drops a heavy dependency from the API image
FastAPI	Flask / Django	Pydantic models generate the OpenAPI spec that generates the TS types — one contract, three consumers
SSE for chat	WebSockets	Unidirectional token streaming. Simpler, survives proxies, works on HF Spaces free tier

04 — Brand system

Sampled directly from `assets/Diplotech_Logo_2.png`. Your current Gradio theme uses emerald and `#4F8EF7` — neither appears anywhere in the DiploTech identity. v3 fixes that.

TOKEN	LIGHT	DARK	ROLE
--brand	#165491	#6BA5E4	Primary actions, links, active nav. The logo's blue bar
--brand-deep	#0F3D6B	#A5C9F0	Hover states, headings on wash
--signal	#C41219	#F0787E	Destructive only — remove book, clear list. The logo's red, darkened for AA
--ink	#12181F	#E3EBF4	Body text. Blue-biased near-black, from the logo's #231F20
--ground	#FAFBFD	#0A0F16	Page background
--surface	FFFFFF	#111925	Cards, panels, sheets
--hairline	#DCE3EC	#22303F	Borders, dividers

Reserve the red. The marketing site uses red as a brand accent; a data-dense product interface cannot. In v3 red means *destructive or failed* and nothing else, so that when it appears the user knows immediately what it means. Brand presence comes from the blue and the logo mark.

Typography

Two families, loaded via `next/font` so they self-host and never flash: a serif for display headings — which reads as institutional and diplomatic rather than generic-startup — and a neutral sans for the interface. A monospace face carries scores, counts, and the pipeline trace.

What carries over from the marketing site

- The bar-mark logo, top-left, linking back to `diplotech-solutions.com`

- Bilingual EN/AR treatment — the app already carries مكتبة إقرأ الرقمية, which should be a first-class part of the masthead, not a README line
- The site's light, high-clarity default — with dark mode added, because this is a tool people use at length, not a page they skim

What deliberately does *not* carry over: the marketing site's hero proportions and CTA rhythm. A product needs to put controls and results above the fold.

05 — The API contract

Every Gradio handler maps to exactly one route. This table is the specification — build the backend against it, generate the types from it.

ROUTE		REPLACES	RETURNS
/api/search	POST	_do_recommend	{ local: Book[], external: Book[], trace: PipelineTrace }
/api/books	GET	_browse	{ items: Book[], page, total, pageCount }
/api/books	POST	_add_book	{ book: Book, indexRebuilt: boolean }
/api/books/{id}	GET	—	Book — new, enables shareable book URLs
/api/books/{id}	DELETE	_remove_book	{ removed: boolean }
/api/books/{id}/similar	GET	_find_similar	Book[]
/api/explain	POST	_ai_explain	{ explanation: string, provider, model }
/api/agent/chat	POST	_chat	SSE stream — tokens, plus tool_call events
/api/agent/thread/{id}	DELETE	_clear_chat	204
/api/reading-list	GET	_load_reading_list	Book[] — no longer HTML
/api/reading-list	POST	_save_to_rl	{ added: boolean, count: number }
/api/reading-list/{id}	DELETE	_rl_remove	{ count: number }
/api/reading-list	DELETE	_rl_clear	204
/api/analytics	GET	_build_analytics	{ ratings: Bin[], categories: Bin[], years: Point[] } — data, not PNGs
/api/export/pdf	POST	_export_results_pdf, _rl_export_pdf	application/pdf
/api/llm/providers	GET	_update_model_choices	{ provider: string, models: string[] }[]
/api/llm/connect	POST	_ai_connect	{ connected, provider, model, message }
/api/llm/status	GET	_initial_ai_status	{ connected, provider?, model?, capabilities }
/api/health	GET	—	{ indexReady, bookCount, agentReady }

The `trace` object is the differentiator

Right now the RAG pipeline's cleverness is invisible — a user searches and gets a list, with no evidence that HyDE ran, that three query variants were fused by RRF, or that a cross-encoder re-ordered the results. v3 returns a trace alongside every search:

```

type PipelineTrace = {
  originalQuery: string
  expandedQuery?: string // llm.expand_query output
  hydeDocument?: string // the hypothetical doc that got embedded
  queryVariants?: string[] // the 3 multi-query rephrasings
  stages: Array<{
    name: "bm25" | "faiss" | "graph" | "rrf" | "rerank"
    candidates: number
    durationMs: number
    topShift?: number // how far re-ranking moved things
  }>
  totalMs: number
}

```

The frontend renders this as a collapsible trace under the search bar. It costs one Pydantic model and one component, and it converts an invisible backend into the most demonstrable thing on the page.

06 — Phased build order

Ordered so that something works end-to-end as early as possible, and so the Gradio app keeps running until the replacement is proven. Each phase ends in a commit on

`feature/v3-typescript-frontend`.

P0 Extract the data layer

DONE

Nothing new gets built until the backend stops emitting HTML. This is the highest-leverage half-day in the whole plan.

- Delete `BookRecommender._render_card`, `_render_section`, `format_books`; `recommend()` already returns dicts, so callers just stop wrapping them
- Delete `ReadingList.to_html()`; keep `get_all()`
- Add a deterministic `id` field in `_normalise()`
- Rewrite `_build_analytics` to return series data; drop Matplotlib from `requirements.txt`
- Update the 81 existing tests to assert on dict shape rather than HTML substrings

SHIP Every core module returns JSON-serialisable data. Gradio still runs, now rendering client-side.

P1 FastAPI service

DONE

- Restructure into `apps/api/` with `core/` holding the existing modules
- Pydantic schemas for `Book`, `SearchRequest`, `SearchResponse`, `PipelineTrace`, `LLMStatus`
- Six routers implementing §05; singletons moved into a `lifespan` handler
- CORS locked to the Vercel origin plus `localhost:3000`
- **Rate limit** the LLM-touching routes — this will be public and keys are yours
- Contract tests via `httpx.AsyncClient` against every route

Checkpoint: `/docs` renders the full OpenAPI spec and every endpoint is exercisable from the browser before a single line of TypeScript is written.

SHIP A running, documented, tested JSON API.

P2 Frontend foundation

~1 day

- `create-next-app` with TypeScript, Tailwind v4, App Router, `strict: true`
- Brand tokens from §04 into `@theme`; `next-themes` for the toggle
- shadcn/ui primitives: button, input, select, slider, switch, dialog, sheet, tabs, badge, skeleton, sonner
- `openapi-typescript` generating `lib/types.ts` from the live spec, wired into `pnpm gen:types`
- App shell: masthead with logo mark, nav, theme toggle, LLM status pill

SHIP Themed, typed, empty app that compiles clean and talks to the API.

P3 Discovery — the flagship screen

~2 days

- `<SearchPanel />` : query, language, category, result counts, min rating, sort, scope, and the four RAG toggles (AI expand · HyDE · multi-query · re-rank)
- `<BookCard />` : cover with skeleton and fallback, title, authors, star rating, source badge, similarity score in mono, save and detail actions
- Results in two labelled groups — local library and external — with a responsive grid
- `<PipelineTrace />` : collapsible stage-by-stage breakdown with timings
- Real empty, loading, and error states; URL-synced search params so results are shareable

SHIP The screen most visitors will judge the project on.

P4 Streaming concierge

~1½ days

- Convert `LibraryAgent.chat()` to an async generator over LangGraph's `astream_events`
- Emit typed SSE frames: `token` , `tool_call` , `tool_result` , `done` , `error`
- `<ChatStream />` renders tokens live and shows *which tool the agent chose* as an inline step — the visible payoff of using LangGraph
- Thread persistence in `localStorage` ; markdown rendering; stop and clear controls

SHIP A chat that streams and shows its reasoning steps.

P5 Catalogue, reading list, analytics, admin

~2 days

- **Catalogue** — virtualised paginated grid, debounced search, filter chips
- **Book detail** at `/book/[id]` — full description, metadata, similar books, save action. New capability; Gradio had no per-book URL
- **Reading list** — optimistic add/remove, PDF and CSV export
- **Analytics** — Recharts rating histogram, category bars, publication-year trend, all theme-aware
- **Admin** — add and remove books with proper form validation and index-rebuild feedback

SHIP Feature parity with v2, plus shareable book pages.

- Dockerfile for the API, pushed to a Hugging Face Space
- Vercel project for `apps/web`, env-wired to the Space
- CI extended: `pytest` · `tsc --noEmit` · `eslint` · `next build`
- Delete `app.py`. Rewrite the README against the new architecture
- Record a short screen capture for the README header

SHIP Two live URLs, green CI, and a README that opens with a demo.

Roughly nine to ten working days at a steady pace. P0 and P1 are the load-bearing ones — if those are done properly, everything after is ordinary frontend work.

07 — Screen inventory

ROUTE	FROM	WHAT CHANGES
/ Discovery	Discovery tab	Sticky filter rail, responsive card grid, live pipeline trace, URL-synced state
/concierge	AI Concierge tab	Token streaming instead of a blocking call; visible tool-selection steps
/catalogue	Catalogue tab	Virtualised list, debounced search, filter chips instead of a page-number box
/book/[id]	NEW	Shareable per-book page with similar-books rail — impossible in Gradio
/reading-list	Reading List tab	Optimistic updates, no full-page refetch on every mutation
/analytics	Analytics tab	Interactive charts with tooltips, replacing static server-rendered PNGs
/admin	Library Admin tab	Validated forms, clear index-rebuild progress
/how-it-works	NEW	The explainer page — see below

08 — Deployment

WHAT “DEPLOYING” MEANS HERE

Putting it on the internet so anyone with the link can use it, without installing anything. Two halves live in two places, because they need different things: the interface is small and needs to load instantly, while the engine is heavy and needs real memory to hold the book index.

Both hosts have free tiers that genuinely suffice, so the running cost is zero. The catch is that free servers fall asleep when unused, and waking this one takes about half a minute — which is why the plan pre-builds the search index into the server package rather than rebuilding it every time it wakes.

FRONTEND · VERCEL

Root `apps/web`. Push-to-deploy, preview URLs per branch, edge CDN, free tier. One env var:

`NEXT_PUBLIC_API_URL`.

BACKEND · HF SPACES

Docker Space on the free CPU tier. Models cached in the image layer. Secrets set in Space settings, never in git.

COST

\$0

per month, both tiers, indefinitely.

THE COLD-START PROBLEM, AND THE FIX

HF free Spaces sleep after inactivity. A cold boot loads `all-MiniLM-L6-v2`, builds the FAISS index over ~6,800 books, fits BM25, and constructs the NetworkX graph — call it 30–45 seconds. A recruiter will not wait that out.

- **Pre-build the artifacts into the image.** Commit or generate `book_index.faiss` and `books_metadata.pkl` at Docker build time rather than on boot. This is most of the win.
- **Cache the knowledge graph** — `hybrid_retriever.py` already has `_graph_cache_path()`; make sure it lands in the image.
- **Pre-download the model** in the Dockerfile so no HF fetch happens at runtime.
- **Warm it from the frontend.** Fire `/api/health` from the root layout on first paint, and show an honest "waking the retrieval engine" state instead of a spinner that looks broken.
- **Optional:** a GitHub Action pinging `/api/health` every 30 minutes keeps it warm for free.

Do not ship your API keys to a public demo. The app supports five providers. For the public deployment, expose Groq (free tier, fast) as the default, rate-limit it per IP, and let visitors paste their own key for Claude or OpenAI. Never commit a key, and rotate anything that has ever been in `.env` before this repo goes fully public.

09 — Making it reviewable

You want people to land on this and understand the system. That is a design problem, and it does not solve itself by having good code.

The `/how-it-works` page

You already have this content — it is spread across `docs/ARCHITECTURE_V2.md`, three standalone HTML files, and two PDFs. In v3 it becomes a page in the app, at a URL, with the real system running underneath it: an interactive walkthrough where a visitor types a query and watches HyDE generate a hypothetical document, sees the three query variants, sees BM25 and FAISS candidate sets side by side, and sees the cross-encoder reorder them. Same data as the trace component, presented pedagogically.

This is the single highest-return thing on the list. It is the difference between "another book search app" and "someone who can explain retrieval architecture to a client."

README, rewritten

- 1. A screen capture GIF above the fold, and the two live links
- 2. One paragraph on what it is, one on why the architecture is what it is
- 3. The architecture diagram — as an SVG, not ASCII art
- 4. A capability table with links into the source for each claim
- 5. Local setup that works from a clean clone in under five commands

Repository hygiene

- Move `INTERVIEW_PREP_ACCENTURE.md` and its PDF out of the public repo — they signal job-hunting rather than engineering
- Same for the loose PDFs at root; link them from `docs/` if they stay
- Delete `archive/`, or move it to a branch
- Remove `API.txt` from the tree and confirm it was never committed with a real key
- Add topics on GitHub: `rag`, `langgraph`, `faiss`, `nextjs`, `typescript`, `fastapi`, `hybrid-search`
- Set the repo description and homepage to the live URL

Branch policy holds. `main` stays as the original version. All of this lands on `feature/v3-typescript-frontend`, with a proper PR against `main` that stays open as a readable record of the migration — which is itself worth showing.

10 — Risks and mitigations

RISK	WHY IT BITES	MITIGATION
Cold starts read as "broken"	Free-tier sleep plus a 30s index build is the most likely reason a reviewer bounces	Bake artifacts into the image; warm on first paint; honest loading copy; optional keep-alive cron
Public LLM key abuse	An open endpoint calling Claude on your key is an uncapped bill	Groq free tier as public default, per-IP rate limit, bring-your-own-key for premium providers
P0 scope creep	Removing HTML rendering touches every test; it is tempting to "improve while in there"	P0 is deletion and shape-change only. No new features until P1 is green
Type drift between API and web	Hand-written TS interfaces silently diverge from Pydantic models	Generate <code>lib/types.ts</code> from OpenAPI; run <code>gen:types</code> in CI and fail on diff
HF Spaces memory ceiling	Free tier is 16 GB but CPU-only; the cross-encoder plus MiniLM plus FAISS plus 6,800 rows is fine, a larger corpus may not be	Lazy-load the re-ranker (already the case), monitor at <code>/api/health</code> , cap corpus growth
Losing working behaviour in translation	Gradio handlers contain accumulated edge-case handling that is easy to drop	Keep <code>app.py</code> in the tree until P6; diff behaviour side-by-side before deleting it
SSE through the Space proxy	Streaming can buffer behind some proxy configurations	Disable response buffering explicitly; fall back to a non-streaming POST if the stream fails to open